

Scenario 3: Shopping Cart

Weak Prompt:

- Test the cart functionality..

Response for Weak Prompt:

Testing shopping cart functionality requires a comprehensive approach to ensure a seamless user experience and accurate backend processing. To approach this systematically—applying standard foundation-level testing principles like Boundary Value Analysis and State Transition testing—here is a structured test plan.

1. Functional Testing (Positive Scenarios)

- **Add Items:** Verify that single and multiple distinct items can be added to the cart successfully.
- **Cart Icon Update:** Ensure the cart counter icon updates immediately when an item is added.
- **Update Quantity:** Verify that the user can increase or decrease the quantity of an item already in the cart, and the total price updates dynamically.
- **Remove Items:** Test removing a single item and clearing the entire cart. Verify the cart reflects an "Empty Cart" message.
- **Calculations:** Validate that the subtotal, taxes, shipping charges, and final total are calculated accurately based on the items present.
- **Coupons/Discounts:** Apply valid promo codes and verify the discount is accurately reflected in the final total.
- **Proceed to Checkout:** Ensure clicking the "Checkout" button redirects the user to the correct payment or shipping page.

2. Negative & Edge Case Testing (Boundary Value Analysis)

- **Exceeding Inventory:** Attempt to add a quantity greater than the available stock. The system should block this and display an appropriate error message.
- **Zero or Negative Input:** Try to manually enter 0, negative numbers, or special characters into the quantity field.
- **Invalid Coupons:** Apply expired or invalid promo codes to ensure the system rejects them gracefully without breaking the price calculation.
- **Simultaneous Updates:** Open the cart in two different tabs. Add an item in one tab and verify the behavior when trying to check out in the other tab.

3. State & Session Management

- **Guest vs. Logged-in User:** Add items as a guest, log into an account, and verify that the cart items are retained and merged with any existing saved cart data.
- **Session Timeout:** Leave items in the cart, allow the session to time out, and verify if the cart retains the items upon logging back in (depending on the specific business requirements).
- **Browser Navigation:** Use the browser's back and forward buttons during the cart and checkout process to ensure the state remains stable and no duplicate orders are created.

4. UI/UX & Compatibility

- **Responsiveness:** Verify the cart's layout, buttons, and text are fully functional and visible on mobile, tablet, and desktop views.
- **Cross-Browser Testing:** Ensure consistent behavior across Chrome, Firefox, Edge, and Safari.
- **Product Links:** Click on the product image or name within the cart to verify it redirects back to the correct product detail page.

5. Performance & Security

- **Load Testing:** Verify the cart's response time when adding items during high-traffic periods.
- **Security:** Attempt basic SQL injection or Cross-Site Scripting (XSS) in text fields like the promo code box to ensure inputs are properly sanitized.

Are you planning to execute these manually, or are you looking to build an automated test framework for this using a language like Python?

ICEPOT Prompt:

- Act as a Senior Manual QA Engineer. Generate manual test cases for the shopping cart functionality of an e-commerce website. Users can add products, update quantities, remove items, and apply discount coupons. Include scenarios for price calculation, quantity updates, coupon validation, cart persistence, out-of-stock products, and item removal. For example, verify that the cart total updates correctly when the quantity of a product is increased. Present the output in a table with Test Case ID, Scenario, Steps, and Expected Result. Use a professional and technical tone.

Response for ICEPOT Prompt:

As a Senior Manual QA Engineer, establishing a robust test suite for the shopping cart functionality is critical for ensuring a frictionless user journey and accurate revenue realization. Below is the manual test execution plan covering the requested functional areas, including state management, boundary conditions, and calculation accuracy.

Shopping Cart Test Execution Plan

Test Case ID	Scenario	Steps	Expected Result
TC-CART-001	Add an in-stock product to an empty cart.	1. Navigate to a Product Details Page (PDP) of an in-stock item. 2. Click the "Add to Cart" button. 3. Open the shopping cart.	The cart badge updates to '1'. The cart UI displays the correct product name, image, unit price, and a quantity of 1.

TC-CA RT-00 2	Verify price calculation upon quantity increase.	1. Open a cart containing one product. 2. Click the '+' button or manually enter '3' in the quantity input field. 3. Trigger the cart update.	The item quantity updates to 3. The line-item total dynamically recalculates (Unit Price $\times$ 3). The cart subtotal, taxes, and final total update accurately.
TC-CA RT-00 3	Verify behavior when entering an invalid quantity.	1. Open a cart containing a product. 2. Enter '0', a negative number (e.g., '-1'), or special characters into the quantity field.	The system rejects the input. Validation prevents the update, and the UI displays an appropriate error message or reverts to the previous valid quantity.

		3. Trigger the cart update.	
TC-CA RT-00 4	Remove an item from a multi-item cart.	1. Open a cart containing at least two distinct products. 2. Click the "Remove" or "Trash" icon adjacent to one of the products.	The selected item is removed from the DOM. The remaining item persists. The subtotal, taxes, and final total recalculate based only on the remaining item.
TC-CA RT-00 5	Apply a valid discount coupon.	1. Open a cart containing products. 2. Enter an active, valid promotional code in the coupon field.	A success message is displayed. The discount amount is explicitly shown as a negative line item. The final order total decreases by the correct percentage or fixed amount.

		3. Click "Apply".	
TC-CA RT-00 6	Apply an invalid or expired discount coupon.	1. Open a cart containing products. 2. Enter an expired or randomly generated string in the coupon field. 3. Click "Apply".	The system rejects the coupon. An inline validation error (e.g., "Invalid or Expired Code") is displayed. The cart total remains unchanged.
TC-CA RT-00 7	Attempt to add an out-of-stock product.	1. Navigate to the PDP of a product with zero inventory. 2. Attempt to click the "Add to Cart" button.	The "Add to Cart" button is disabled or replaced with an "Out of Stock" state. The user is prevented from adding the item to the cart payload.

TC-CA RT-00 8	Exceed available inventory via quantity update.	1. Add an item with a known stock limit (e.g., 5 items left). 2. Navigate to the cart and update the quantity to 6. 3. Trigger the update.	The system prevents the update. An error message (e.g., "Only 5 items available in stock") is displayed, and the quantity reverts to the maximum allowable limit.
TC-CA RT-00 9	Verify cart persistence across sessions (Logged-in User).	1. Log into a user account. 2. Add items to the cart. 3. Log out, clear local storage/cookies, and log back in.	The cart state is fetched from the backend database. The items added prior to logging out are fully restored with their correct quantities and prices.

		4. Navigate to the cart.	
TC-CART-010	Verify cross-tab synchronization.	1. Open the application in Tab A and add an item to the cart. 2. Open the application in Tab B and add a different item to the cart. 3. Return to Tab A and proceed to checkout.	The cart state synchronizes across both instances. Tab A's checkout payload includes both items (the item added in Tab A and the item added in Tab B).